

Let's make the ultimate power move and shift your title from Cloud Administrator to **Automation Engineer**.

We are going to install **Terraform** on your local **Debian 13 builder workstation**, configure it to securely talk to AWS, and initialize your project workspace. By the end of this task, you will be completely ready to spin up infrastructure using code.

Step 1: Install Terraform on Debian 13

Because Terraform isn't always in the default Debian repositories, we need to add the official HashiCorp repository. Open your Debian 13 terminal and run the following commands:

1. **Install the prerequisites:**

```
Bash
```

```
sudo apt-get update && sudo apt-get install -y gnupg software-properties-common curl
```

2. **Add the HashiCorp GPG Key:**

```
Bash
```

```
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o  
/usr/share/keyrings/hashicorp-archive-keyring.gpg
```

3. **Add the official HashiCorp Repository:**

```
Bash
```

```
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]  
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee  
/etc/apt/sources.list.d/hashicorp.list
```

4. **Update packages and Install Terraform:**

```
Bash
```

```
sudo apt-get update && sudo apt-get install terraform
```

5. **Verify the installation:**

```
Bash
```

```
terraform -help
```

(If it prints out a list of commands, Terraform is successfully installed!)

Step 2: Set Up Your Project Directory

Good engineers keep their code organized. Let's create a dedicated workspace folder for your Infrastructure as Code.

1. **Create and enter your workspace folder:**

```
Bash
```

```
mkdir -p ~/projects/dev-secure-cloud  
cd ~/projects/dev-secure-cloud
```

2. **Create your primary configuration files:**

Terraform looks for files ending in .tf. Let's create the foundational files using the touch command:

```
Bash
```

```
touch providers.tf main.tf variables.tf outputs.tf
```

Step 3: Define Your Cloud Provider

Open the providers.tf file using your favorite terminal text editor (like nano or vim):

```
Bash
```

```
nano providers.tf
```

Paste the following configuration into it. This tells Terraform that it needs to download the specific AWS plugin to communicate with your account:

```
Terraform
```

```
terraform {  
  required_version = ">= 1.5.0"  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}  
  
provider "aws" {  
  region = "us-east-1" # Your primary lab region  
}
```

Save and exit the file (Ctrl+O, Enter, Ctrl+X if using nano).

Step 4: Connect Terraform to Your AWS CLI Credentials

Terraform automatically detects the AWS credentials you set up previously via the AWS CLI. To verify that it can read your study-admin environment, make sure you have initialized your credentials in this terminal session:

```
Bash
```

```
aws sts get-caller-identity
```

If this command correctly returns your study-admin user details, Terraform is fully authorized to act on your behalf.

Step 5: The Initialization (The Grand Moment)

Now, let's run the command that pulls everything together. While inside your ~/projects/dev-secure-cloud directory, run:

```
Bash
```

```
terraform init
```

What to look for:

Terraform will read your providers.tf file, reach out to the internet, and download the official AWS provider plugin into a hidden directory (.terraform/).

Look for the glorious green message: **"Terraform has been successfully initialized!"**

Run through these steps on your Debian machine. Once you get that successful initialization message, paste a screenshot or drop a message, and we will write the actual code to spin up **dev-custom-vpc!**